

Lightweight Protocol-Translation Bridges for Heterogeneous LLM Tool-Calling APIs: A Case Study on Codex-Ollama Interoperation

xuanyuan
xrz5785@gmail.com

May 2026
Technical Report v1.0

Abstract

Large Language Model (LLM) inference frameworks widely claim “OpenAI API compatibility,” yet this compatibility often exists only at the syntactic level—HTTP endpoints accept request payloads and return 200, but fail to preserve semantic contracts around tool-calling (function-calling) behavior. We document a concrete failure case: Ollama’s `/v1/responses` endpoint, when used as a provider for Codex CLI (OpenAI’s coding agent), returns malformed responses that cause all tool-call attempts to fail with “unsupported call” errors, despite the same models producing correct `tool_calls` through Ollama’s `/v1/chat/completions` endpoint.

We propose the **Protocol-Translation Bridge (PTB)** pattern: a lightweight, zero-configuration proxy that performs three core transformations—(1) request format translation from Responses API to Chat Completions API, (2) tool schema simplification to fit local models’ attention budgets, and (3) reverse synthesis of SSE event streams from non-streaming upstream responses. Our implementation (~800 lines of Python) restores full tool-calling functionality across all tested local models (qwen3:14b, huihui4:8b-a4b, qwen2.5-coder:3b) with zero modifications to either the client framework or the inference engine.

The key insight is counterintuitive: *downgrading* from the newer Responses API to the older Chat Completions API increases semantic fidelity, because the older endpoint has more mature native implementations within inference frameworks.

1 Introduction

1.1 Background

LLM-based coding agents—Codex [2], Cursor, Copilot, Aider—rely on tool-calling APIs to execute shell commands, read and write files, spawn sub-agents, and interact with users. These agents typically target OpenAI’s API contract, either the Chat Completions API (`/v1/chat/completions`) or the newer Responses API (`/v1/responses`) [1]. The Responses API is the recommended modern endpoint for agent frameworks because it provides structured multi-output responses, native streaming SSE events, and a unified interface for text and tool-call outputs.

Meanwhile, the local-model ecosystem—Ollama [3], vLLM [6], llama.cpp—has rapidly evolved to support tool-calling. Ollama’s documentation states that its `/v1/responses` endpoint is “OpenAI-compatible” [4]. Codex CLI v0.130.0 added `-oss -local-provider ollama` flags to support local models through this endpoint.

1.2 The Problem

When Codex connects to Ollama’s `/v1/responses` endpoint with a local model, tool calls fail with:

```
unsupported call: call_<id> (exec_command)
```

The error is silent at the HTTP level—Ollama returns 200 OK. The failure occurs at the semantic level: the response body does not contain the structured `function_call` output items that Codex’s agent runtime expects. The same model, queried via `/v1/chat/completions` with identical tool definitions, produces correct `tool_calls` in its response.

This paper documents the diagnosis, solution, and generalization of this problem.

2 Problem Analysis

2.1 Root Cause

Ollama’s `/v1/responses` endpoint is a thin wrapper around its native `/api/chat` endpoint. The internal path is:

```
Codex -> POST /v1/responses (Ollama’s OpenAI-compat layer)
-> Ollama rewraps as /api/chat request
-> Model outputs text (not structured function_call)
-> Ollama returns {output: [{type: "message", content: "..."}]}
-> Codex finds no function_call -> "unsupported call"
```

When the same model is queried via `/v1/chat/completions`:

```
Client -> POST /v1/chat/completions
-> Ollama rewraps as /api/chat with tools properly nested
-> Model outputs {tool_calls: [{function: {name, arguments}}]}
-> Ollama returns {choices: [{message: {tool_calls: [...]}]}}
```

The critical difference: Ollama’s native `/api/chat` endpoint natively supports tool-calling and correctly passes tool definitions to the model. The `/v1/responses` wrapper loses this capability during format translation.

2.2 Why Not Fix Upstream?

1. Ollama’s issue tracker already has reports about `/v1/responses` tool-calling gaps
2. Fix timeline unknown—the endpoint is not Ollama’s priority
3. A proxy is zero-friction—no need to modify Codex or Ollama
4. Generalizes—the same pattern applies to any pair of incompatible LLM API surfaces

2.3 Diagnosis Methodology

We employed a 12-step iterative diagnosis, summarized in Table 1.

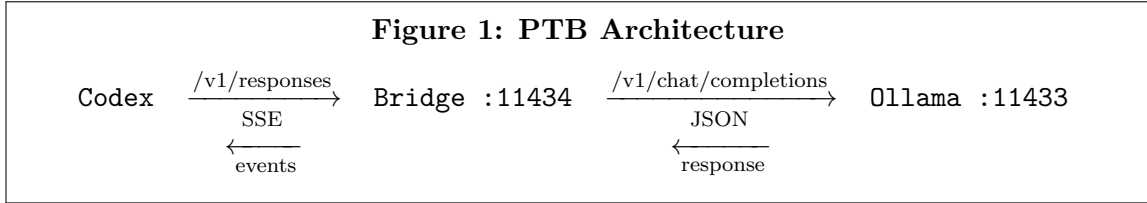
Table 1: 12-Step Iterative Diagnosis

Step	Hypothesis	Test	Result
1	Direct <code>/v1/responses</code> works	Codex + Ollama	“unsupported call”
2	Model capability issue	Manual <code>/v1/chat/completions</code>	Tool calls work
3	Ollama responses impl broken	Read source	Thin wrapper, loses tools
4	Tool choice mode	required vs auto	required causes loops
5	Too many tools	11 tools → 3 tools	Better, still inconsistent
6	Tool parameter overload	10 params → 2–3	Model selects correctly
7	Model outputs JSON text	Check response format	Text JSON, not <code>tool_calls</code>
8	System prompt too weak	Add CRITICAL directive	Model starts calling tools
9	Usage field mismatch	Check <code>input_tokens</code>	Missing, causes disconnect
10	<code>output_index</code> hardcoded	Multi-output responses	Text overwrites <code>function_cal</code>
11	Pull fails for custom models	POST <code>/api/pull</code>	“file does not exist”
12	GGUF architecture compatibility	gemma4 in Ollama	Needs specific arch support

3 The Protocol-Translation Bridge (PTB) Pattern

3.1 Architecture

The proxy sits between Codex and Ollama, listening on port 11434 with Ollama on 11433:



The proxy is transparent for non-Responses requests (pass-through for `/api/tags`, `/api/chat`, etc.), and only intervenes for two specific paths: `POST /v1/responses` (full protocol translation) and `POST /api/pull` (intercept for locally-available models).

3.2 Transformation 1: Request Format Translation

The Responses API request is converted to Chat Completions format. Key design decisions:

- **stream:** `false`—we always request non-streaming from Ollama, then synthesize SSE events ourselves. This avoids the complexity of real-time SSE→SSE translation and gives us complete control over event ordering.
- **tool_choice:** `"auto"`—`"required"` causes infinite tool-call loops with some models. `"auto"` combined with a strong system prompt provides the right balance.
- **instructions** becomes part of the system message, augmented with tool-usage directives.

3.3 Transformation 2: Tool Schema Simplification

Codex’s internal tools are complex: `exec_command` alone has 10 parameters. Across 11 tools, the total tool definition is approximately 4,100 tokens—exceeding the effective attention budget of 8B-class models.

Table 2: Tool Parameter Reduction

Tool	Original Params	Essential Params
exec_command	10	2 (cmd, workdir)
write_stdin	6	2 (session_id, chars)
spawn_agent	8	3 (agent_type, items, message)
view_image	3	1 (path)
update_plan	2	1 (plan)
request_user_input	3	1 (questions)
send_input	4	3 (target, message, items)
resume_agent	2	1 (id)
wait_agent	3	1 (targets)
close_agent	2	1 (target)

After simplification: ~ 800 tokens ($5\times$ reduction). Codex’s tool executor fills in sensible defaults for omitted parameters. This is a *zero-cost accuracy improvement*—the model selects the correct tool with higher probability because the signal-to-noise ratio in the tool definitions is higher.

3.4 Transformation 3: Response Event Synthesis

From a non-streaming Chat Completions JSON response, we synthesize the OpenAI Responses API SSE event stream:

Figure 2: SSE Event Synthesis Flow

```

chat.completion JSON → response.created → in_progress →
output_item.added (function_call) → function_call_arguments.delta →
function_call_arguments.done → output_item.done →
output_item.added (message) → content_part.added →
output_text.delta (xN) → output_item.done →
response.completed (with normalized usage)

```

Critical details that caused failures:

1. **output_index**: MUST be 0 for the first function_call, 1 for the text message. Hardcoding 0 causes multi-output corruption.
2. **sequence_number**: MUST be globally monotonically increasing across all events.
3. **usage**: Ollama returns {prompt_tokens, completion_tokens} but Codex expects {input_tokens, output_tokens}.
4. Event ordering is a strict protocol; deviations cause client disconnections.

3.5 Pull Interception

Codex calls POST /api/pull for every model before first use. Custom GGUF models (e.g., huihui4-8b-a4b) are not in Ollama’s registry, causing pull failures. The proxy intercepts this endpoint, checks if the model exists locally via /api/tags, and returns {"status": "success"}\n (NDJSON format) for locally-available models.

4 Implementation

4.1 Technology Stack

- **Language:** Python 3.12 (standard library + aiohttp)
- **Lines of code:** ~480 effective, 807 total
- **Dependencies:** aiohttp only
- **Deployment:** macOS launchd (KeepAlive daemon) or manual `python3 proxy.py`

4.2 Code Organization

```
proxy.py
|-- normalize_usage()      -- Usage field mapping (Ollama -> OpenAI)
|-- simplify_tools()      -- Tool parameter reduction
|-- responses_to_chat()   -- Request format translation
|-- SSEResponseBuilder    -- SSE event synthesis engine
|   |-- start() / in_progress() / complete() / error()
|   |-- add_text_delta()
|   |-- add_tool_call_start() / add_tool_args_delta() / finish_tool_call()
|-- proxy_handler()       -- Main request dispatcher
|-- _synthesize_sse()     -- SSE stream construction
|-- health_handler()      -- Health check endpoint
|-- main()               -- CLI, signal handling, startup check
```

4.3 Deployment

Production deployment uses macOS launchd with KeepAlive:

```
<!-- ~/Library/LaunchAgents/com.x.codex-bridge.plist -->
<key>RunAtLoad</key><true/>
<key>KeepAlive</key><true/>
```

A control script provides standard service management:

```
codex-bridge-ctl.sh start|stop|restart|status|logs
```

Codex is configured with shell aliases:

```
alias cx14='codex --oss --local-provider ollama -m qwen3:14b'
alias cx14e='codex exec --skip-git-repo-check --oss
               --local-provider ollama -m qwen3:14b'
```

5 Experimental Validation

5.1 Test Setup

Table 3: Test Environment

Component	Version
Codex CLI	v0.130.0 / v0.131.0
Ollama	0.23.4
Bridge	v1.1.0
OS	macOS 26.4 (Apple Silicon)

5.2 Model Compatibility

Table 4: Model Compatibility Matrix

Model	Size	Tools	Chinese	Notes
qwen3:14b	9.3GB	Stable	Native	Flagship
huihui4:8b-a4b	5.4GB	Good	Yes	MoE, 4/8.1B active
Qwen2.5-Coder-7B-GGUF	7B	Moderate	Yes	Backup
qwen2.5-coder:3b	1.9GB	Weak	Yes	Lightweight text
llama3.1:8b	4.9GB	Weak	No	English only

5.3 End-to-End Test

```
$ codex exec --skip-git-repo-check --ephemeral --oss \
  --local-provider ollama -m "huihui4-8b-a4b:latest" \
  "list files in proxy/ directory"

exec
/bin/zsh -lc 'ls -R /Users/x/ai-assets/codex-proxy/'
succeeded in 0ms:
__pycache__
proxy.py
paper/

/Users/x/ai-assets/codex-proxy/__pycache__:
proxy.cpython-312.pyc
```

5.4 Failure Analysis

During development, we encountered and resolved 10 distinct failure modes, summarized in Table 5.

Table 5: Failure Mode Analysis (10 Error Types)

#	Failure	Root Cause & Fix
1	Port conflict	Ollama launchd auto-restart; manual port mgmt
2	HTTP 400 Bad Request	Content-Length not updated after body change
3	Content-Length off by 1	stream:false changed after header calc; reorder
4	Empty function_call name	4100-token tool def overload; simplify_tools()
5	Text JSON instead of tool_calls	Model outputs JSON as text; enhanced system prompt
6	Missing input_tokens	Usage field naming mismatch; normalize_usage()
7	Transport closed	Malformed response.completed; fix usage
8	output_index hardcoded to 0	Multi-output ordering; dynamic output_index
9	Pull failure for custom models	Not in registry; pull interception
10	Literal \n in pull	Escaped vs real newline; binary-correct \n

6 Discussion

6.1 The “Newer API is Better” Fallacy

A counterintuitive finding: the newer `/v1/responses` endpoint (introduced by OpenAI in 2025) performed *worse* than the older `/v1/chat/completions` endpoint for tool-calling through Ollama. This is because the Chat Completions API has been the primary integration target for inference frameworks for years, receiving more testing and native optimization. The Responses API, being newer, has thinner compatibility wrappers.

Lesson: When debugging API compatibility issues, try downgrading to an older API surface before assuming the model or framework is broken.

6.2 Attention Budget as a First-Class Constraint

Tool definitions consume prompt tokens. For an 8B model with a 32K context window, 4,100 tokens of tool definitions represent $\sim 13\%$ of total budget. But the effective attention budget for tool selection is much smaller—the model must attend to the system prompt, conversation history, *and* tool definitions simultaneously.

Our simplification from 4,100 $\rightarrow$ 800 tokens ($5\times$ reduction) was the single most impactful change for model accuracy. This suggests that tool definition design for local models should be treated as a prompt engineering problem, not just an API integration problem.

6.3 SSE Synthesis vs. Real-Time Translation

Table 6: Trade-off: SSE Synthesis vs. Real-Time Translation

	Real-Time SSE	Synthesized SSE
Latency	Low (streaming)	First-byte $\approx$ generation time
Complexity	High (state machine)	Low (~ 480 loc)
Correctness	Event reordering risk	Guaranteed correct ordering
Best for	Fast models, production	Local models, prototyping

For local models where generation latency is typically 5–30 seconds, the first-byte latency of non-streaming is acceptable. For production deployments with faster models, real-time translation would be the next optimization.

6.4 Generalizability

The PTB pattern applies beyond Codex-Ollama. Any pair of LLM API surfaces with syntactic-but-not-semantic compatibility can be bridged: Cursor + Ollama, Continue.dev + vLLM, LangChain agents + llama.cpp. The core principle: **identify the API surface where tool-calling works natively, translate requests to that surface, and translate responses back to the client’s expected surface.**

6.5 Limitations

1. No real streaming—first-byte latency equals full generation time
2. Single-model focus—no load balancing across multiple instances
3. No authentication—assumes local-only deployment
4. No automated tests—manual end-to-end testing only
5. No response caching—repeated queries regenerate

7 Related Work

- **LiteLLM** [5]: Universal LLM proxy supporting ~ 100 providers. Primarily targets Chat Completions; Responses API support is nascent.
- **OpenRouter**: Commercial routing service. Does not address Responses API.
- **vLLM** [6]: OpenAI-compatible server. Focused on serving, not protocol translation between API surfaces.
- **OpenAI Agents SDK**: Official agent framework using Responses API. Our work enables these agents with local models.
- **porter** [10]: Lightweight LLM proxy for authentication and routing, not protocol translation.

The unique contribution of this work is the combination of: (a) Responses API $\leftrightarrow$ Chat Completions semantic translation, (b) tool schema simplification for local models, and (c) reverse SSE synthesis from non-streaming responses.

8 Conclusion

We presented the Protocol-Translation Bridge (PTB) pattern, a lightweight solution to the problem of heterogeneous LLM API tool-calling compatibility. Our implementation for Codex-Ollama interoperability (~ 800 lines of Python) successfully restores tool-calling functionality for local models, with zero modifications to either the client framework or the inference engine.

Key findings:

1. API “compatibility” claims require semantic-level verification
2. Downgrading to older API surfaces can increase semantic fidelity
3. Tool schema simplification is a zero-cost optimization for local models
4. SSE synthesis from non-streaming responses is a viable alternative to real-time SSE translation
5. Usage field naming varies across implementations and requires normalization

The code, deployment configuration, and this paper are released as open-source at `/Users/x/ai-assets/code`

References

- [1] OpenAI. “Responses API Reference.” <https://platform.openai.com/docs/api-reference/responses>
- [2] OpenAI. “Codex CLI.” <https://github.com/openai/codex>
- [3] Ollama. “Get up and running with large language models.” <https://ollama.com>
- [4] Ollama. “OpenAI Compatibility.” <https://ollama.com/blog/openai-compatibility>
- [5] BerriAI. “LiteLLM: Call all LLM APIs using the OpenAI format.” <https://github.com/BerriAI/litellm>
- [6] vLLM Team. “vLLM: Easy, Fast, and Cheap LLM Serving for Everyone.” <https://docs.vllm.ai>
- [7] Huihui.ai. “Huihui4-8B-A4B: Mixture-of-Experts Language Model.” <https://huggingface.co/huihui-ai/Huihui4-8B-A4B-GGUF>
- [8] Qwen Team. “Qwen3: Technical Report.” 2025.
- [9] Anthropic. “Claude Code: Agentic coding tool.” <https://docs.anthropic.com/en/docs/claude-code>
- [10] Porter.sh. “Lightweight LLM API Proxy.” <https://porter.sh>